



جامعة الكنوز
**AL- KUNOOZE
UNIVERSITY**

Department of Cyber Security Science

Report of: Computer Programming II

(CyS101) Class

Second semester (First Stage)

Title: **Bank Simulation System**

Name: **Ali Alkarrar Abdullah Aboud**

Date: 2025/04/05

Abstract

This report describes the development of a Bank Simulation System using Object-Oriented Programming (OOP) principles. The system simulates core banking operations, including account creation, deposits, withdrawals, and balance checks. The objective is to demonstrate how OOP concepts, particularly encapsulation, can be effectively applied to model real-world banking scenarios. The BankAccount class was central to the implementation, encapsulating the account holder's information and balance, while providing methods to perform various banking operations. The report discusses the design, functionality, and implementation of the system, highlighting how OOP enhances the flexibility and maintainability of the code.

Table of contents

<i>Introduction</i>	<i>Page(2)</i>
<i>Methodology (Python Code)</i>	<i>Page(3)</i>
<i>Results (Running of Python Code)</i>	<i>Page(7)</i>
<i>Conclusion/recommendations (future work)</i>	<i>Page(8)</i>
<i>Conclusion.....</i>	<i>Page(9)</i>
<i>References</i>	<i>Page(9)</i>

I. Introduction

This report presents a Bank Simulation System built using OOP principles.

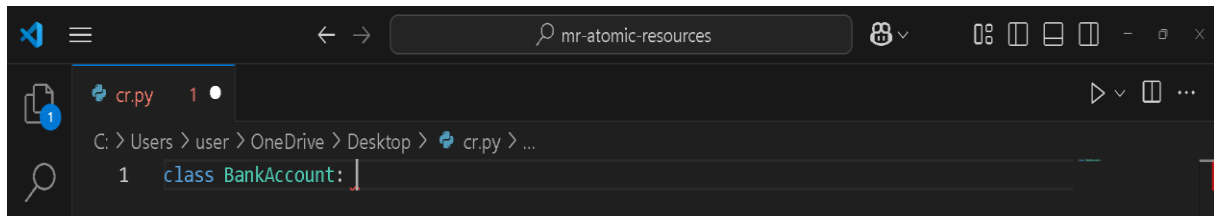
The system simulates basic banking operations, such as creating accounts, depositing and withdrawing money, and checking balances. The goal of this simulation is to provide a practical example of how OOP can be applied to model real-world scenarios, with a focus on the BankAccount class and its interactions.

Through this simulation, we explore the key concepts of OOP, such as encapsulation, and demonstrate how they can be implemented to manage customer accounts in a banking system.

II. Methodology (Python Code)

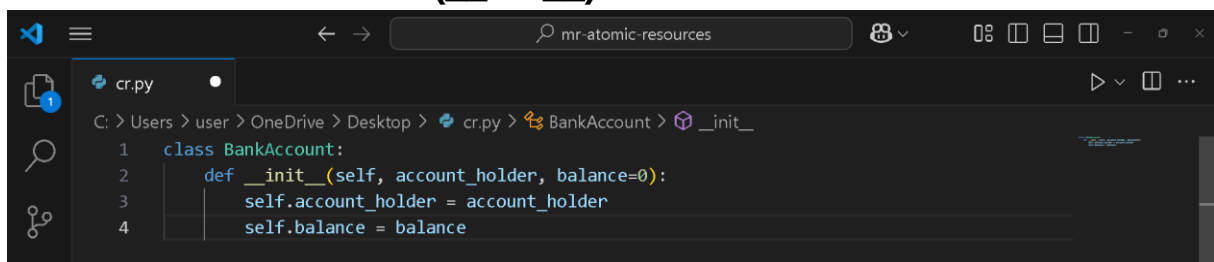
Explanation of the Code

1. BankAccount class definition:



- Here, we create a class called BankAccount that represents a bank account.
- The class contains the data (attributes) and methods for the account.

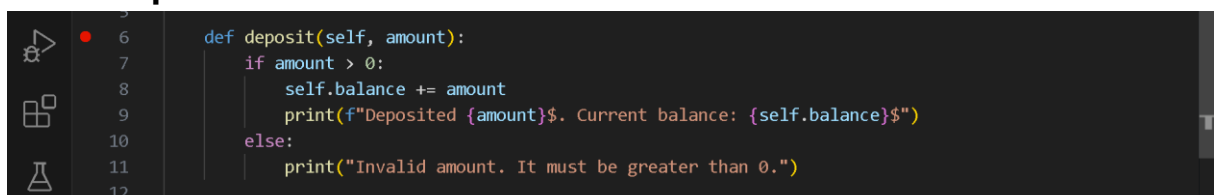
2. initializer function (__init__):



This function is used to specify basic properties when creating a new account.

- **account_holder:** The name of the account holder.
- **balance:** The balance, starting with 0 if not specified.
- **self:** Indicates that these variables are specific to this object (account).

3. Deposit function:



- This function allows you to deposit an amount of money into the account.
- The amount is first checked to be greater than zero.

- If it is correct, the amount is added to the balance and a message is printed.
- If it is incorrect, an error message is printed.

4. Pull function (withdraw):

```

12
13 def withdraw(self, amount):
14     if amount > 0 and amount <= self.balance:
15         self.balance -= amount
16         print(f"Withdrew {amount}$. Current balance: {self.balance}$")
17     else:
18         print("Insufficient balance or invalid amount.")
19

```

- This function allows you to withdraw an amount from the account.
- The requested amount is checked to be greater than zero and less than or equal to the balance.
- If the conditions are met, the amount is deducted from the balance.
- If the conditions are not met, a message is printed indicating that the operation is not possible.

5. Balance check function(check_balance):

```

19
20 def check_balance(self):
21     print(f"Current balance for {self.account_holder}: {self.balance}$")
22
23
24
25
26

```

- This function prints the current account balance of the account holder.
- It displays the account holder's name and the current balance.

6. Program completion (main logic):

What does this part add?

- Allows the user to enter their name.
- Displays a list of options: Deposit, Withdraw, Check Balance, Exit.
- Interacts with the user using Input.
- Uses a while True loop to iterate through the list until the user selects "Exit."

Python code:

```
cr.py
C: > Users > user > OneDrive > Desktop > cr.py > ...

1 class BankAccount:
2     def __init__(self, account_holder, balance=0):
3         self.account_holder = account_holder
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             print(f"Deposited {amount}$. Current balance: {self.balance}$")
10        else:
11            print("Invalid amount. It must be greater than 0.")
12
13    def withdraw(self, amount):
14        if amount > 0 and amount <= self.balance:
15            self.balance -= amount
16            print(f"Withdrew {amount}$. Current balance: {self.balance}$")
17        else:
18            print("Insufficient balance or invalid amount.")
19
20    def check_balance(self):
21        print(f"Current balance for {self.account_holder}: {self.balance}$")
22
23 def main():
24     print("Welcome to the Bank Simulation System!")
25     name = input("Enter account holder name: ")
26     account = BankAccount(name)
27
28     while True:
29         print("\n--- Menu ---")
30         print("1. Deposit")
31         print("2. Withdraw")
32         print("3. Check Balance")
33         print("4. Exit")
34
35         choice = input("Choose an option (1-4): ")
36
37         if choice == '1':
38             try:
39                 amount = float(input("Enter amount to deposit: "))
40                 account.deposit(amount)
41             except ValueError:
42                 print("Please enter a valid number.")
43
44         elif choice == '2':
45             try:
46                 amount = float(input("Enter amount to withdraw: "))
47                 account.withdraw(amount)
48             except ValueError:
```

supplement... ↓

```
48         except ValueError:
49             print("Please enter a valid number.")
50
51     elif choice == '3':
52         account.check_balance()
53
54     elif choice == '4':
55         print("Exiting. Thank you for using our system.")
56         break
57
58     else:
59         print("Invalid option. Please choose between 1 and 4.")
60
61 if __name__ == "__main__":
62     main()
63
```

III. Results (Running of Python Code)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\user\OneDrive\Desktop\mr-at
:/Users/user/OneDrive/Desktop/cr.py
PS C:\Users\user\OneDrive\Desktop\mr-at
:/Users/user/OneDrive/Desktop/cr.py
Welcome to the Bank Simulation System!
Enter account holder name: █
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

:/Users/user/OneDrive/Desktop/cr.py
Welcome to the Bank Simulation System!
Enter account holder name: ali alkarrar

--- Menu ---
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): █
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Choose an option (1-4): 1
Enter amount to deposit: 500
Deposited 500.0$. Current balance: 500.0$

--- Menu ---
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): █
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Deposited 300.0$. Current balance: 800.0$

--- Menu ---
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): 3
Current balance for ali alkarrar: 800.0$
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

Current balance for ali alkarrar: 800.0$

--- Menu ---
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Choose an option (1-4): 4
Exiting. Thank you for using our system.
```


Conclusion/recommendations (future work)

In this project, we developed a simple yet functional Bank Simulation System using Object-Oriented Programming in Python. The system allowed users to perform key operations such as creating accounts, depositing funds, withdrawing money, and checking balances through a command-line interface. The use of a **BankAccount** class demonstrated how OOP principles, particularly encapsulation, can be effectively applied to manage real-world scenarios in a clean and organized manner.

For future improvements, the system can be extended to include features such as:

- **User authentication** (e.g., using usernames and passwords).
- **Transaction history logs** for each account.
- **Error logging and exception handling** for more robust performance.
- **Support for multiple accounts and account types** (e.g., savings, checking).
- **A graphical user interface (GUI)** to enhance usability.

These enhancements would not only increase the system's functionality but also bring it closer to the structure and behavior of real-world banking applications.

Conclusion

This project involved the development of a basic Bank Simulation System using Object-Oriented programming principles. It allowed for key banking operations like deposits, withdrawals, and balance checks. By creating the BankAccount class, we were able to demonstrate how OOP concepts like encapsulation can improve code organization and reusability.

Although simple, the system provides a clear understanding of the fundamental structure needed for more complex banking applications, reinforcing key programming concepts that will aid in future software development.

References

-  **Python Official Documentation – Classes**
-  **ChatGPT and DeepSeek**
-  **W3Schools - Python OOP**